



Секреты отладки производительности и ускорения YDB – распределенной СУБД на C++

Евгений Иванов,

Ведущий разработчик YDB, Яндекс

Евгений Иванов

Ведущий разработчик YDB,
Яндекс



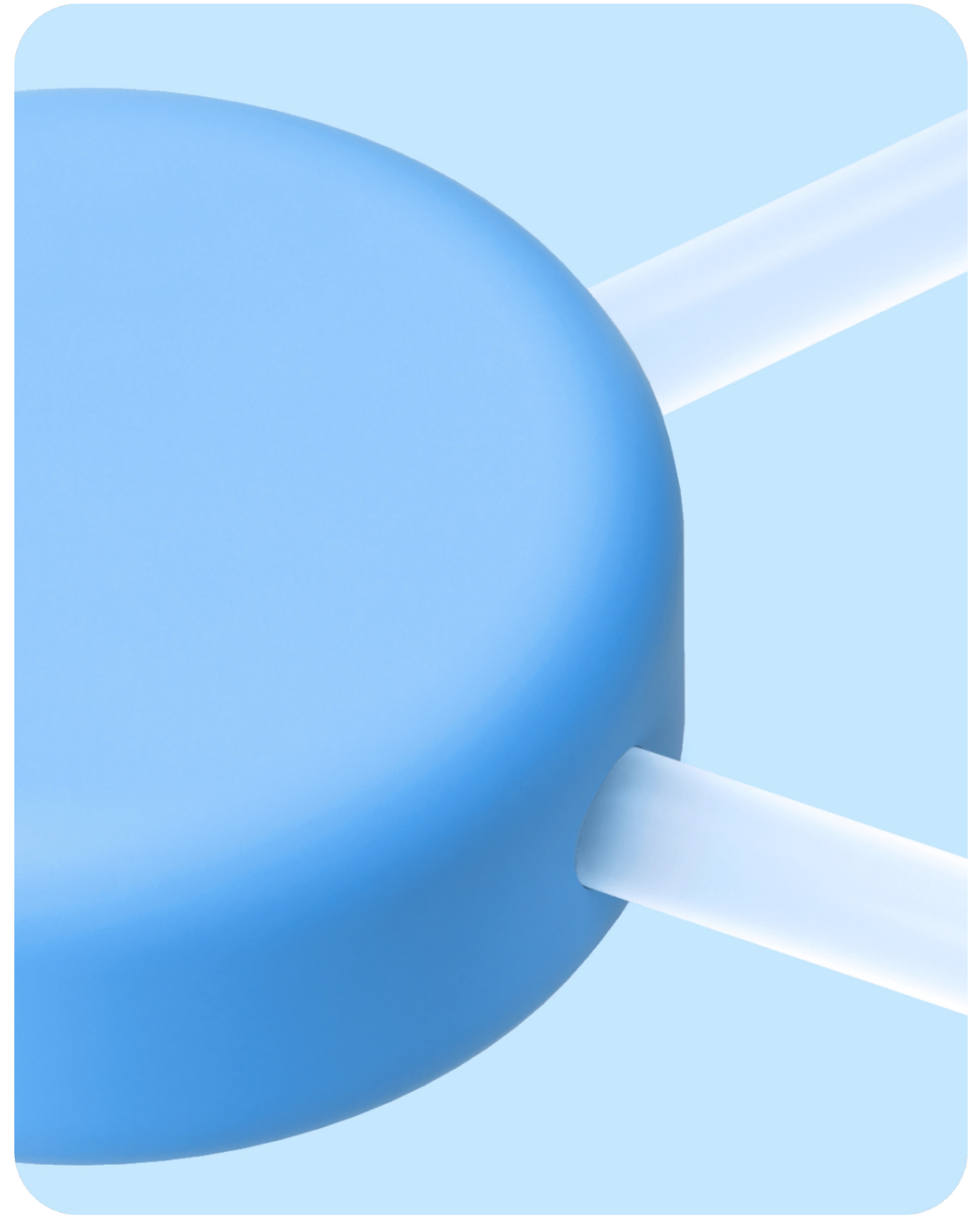
Александр Крюков

Старший разработчик YDB,
Яндекс



Почему YDB на конференции по C++?

01



YDB – распределенная платформа, реализованная на C++

1

**Изначально
OLTP**

2

**YDB Topic
Service
(аналог kafka)**

**Транзакции
между
топиками
и таблицами**

3

**Недавно
добавили
поддержку
OLAP**

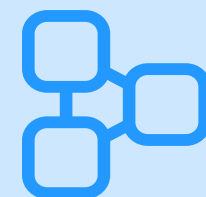
4

И не только

Строгая консистентность

1

**CAP-теорема —
выбираем CP**



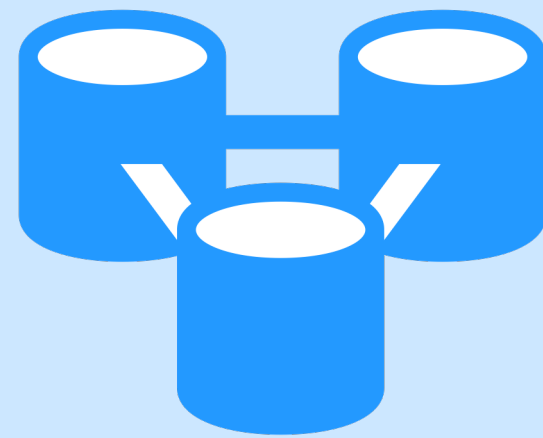
2

**Serializable уровень
изоляции транзакций**



Высокая доступность и отказоустойчивость

Может работать
в нескольких ДЦ

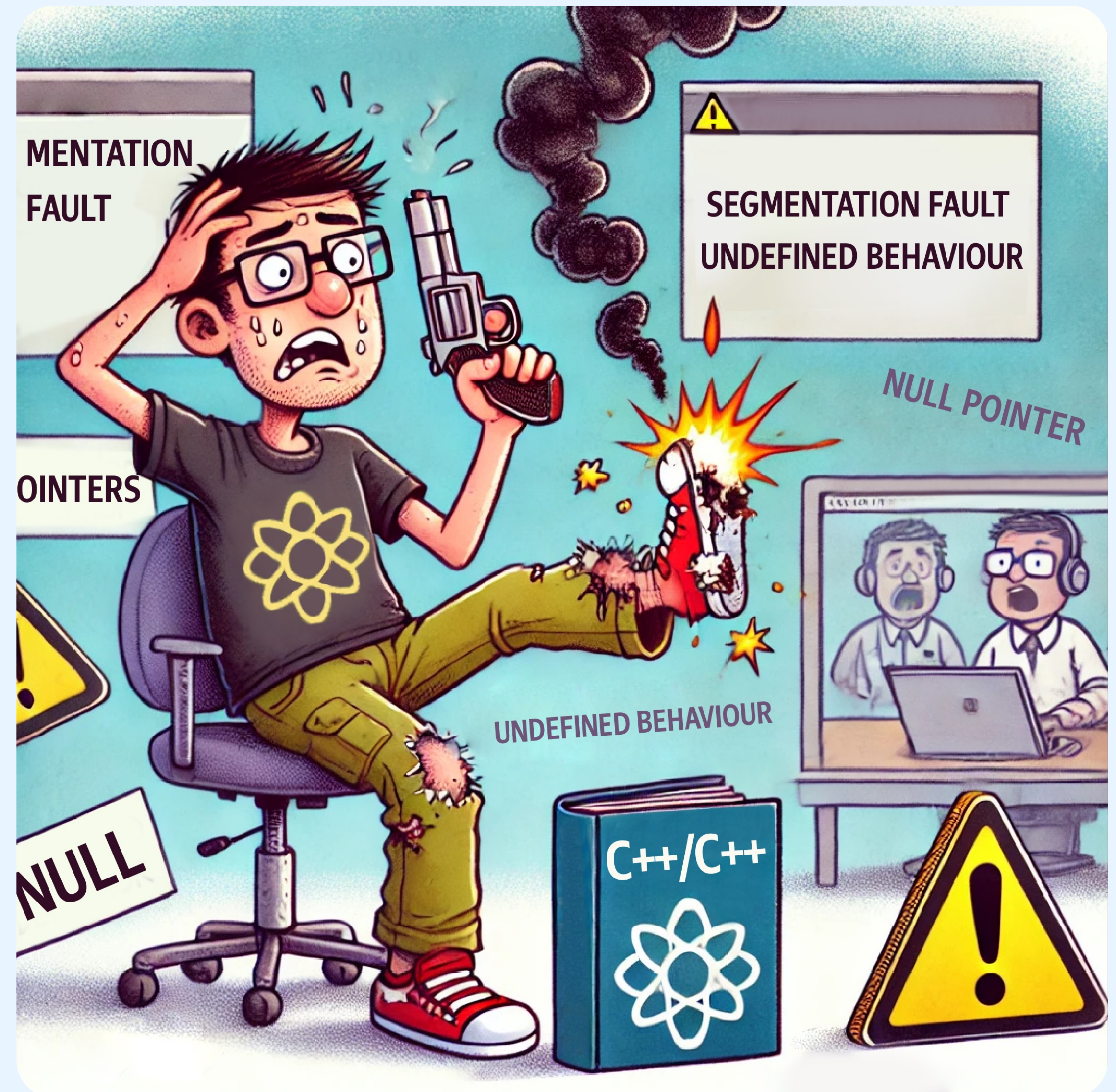


YDB переживает
одновременный отказ ДЦ
и стойки в другом ДЦ

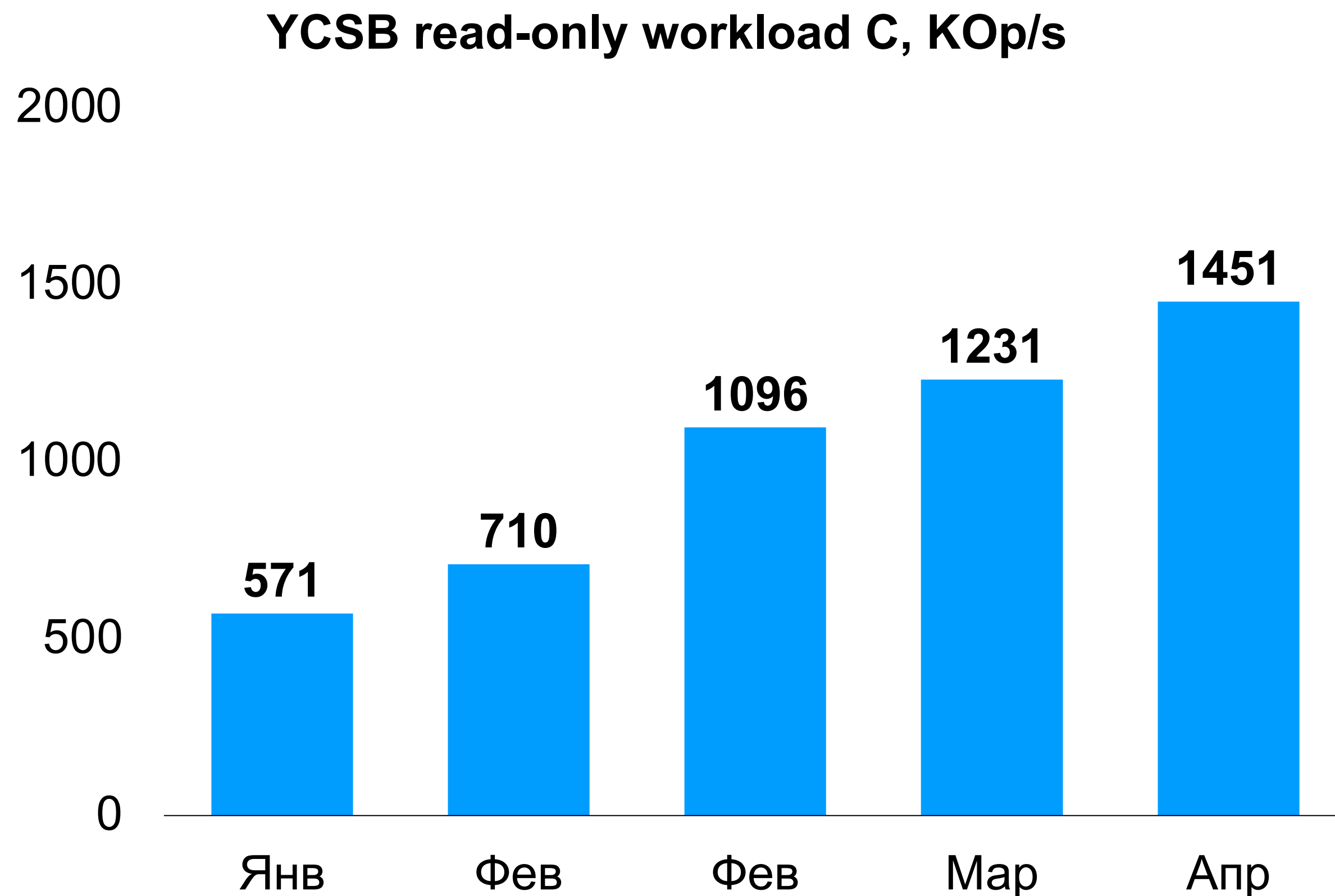


Почему C++?

- Первый коммит YDB более 10 лет назад
- В Яндексе C++ был и остается основным языком для высоконагруженных проектов
- Отсутствие сборщика мусора
- Прямое управление памятью
- Zero-cost abstraction



Поиск и устранение узких мест в YDB в 2023 г.



- 128 ядер: два Intel Xeon Gold 6338 2,00 ГГц с hyperthreading
- 4 NVMe Intel-SSDPE2KE032T8
- 512 ГБ RAM
- transparent huge pages
- Ubuntu 20.04.3 LTS

Доклад о том, как мы ускорились в несколько раз

1

**Как искали узкие места
в большом и сложном проекте**

2

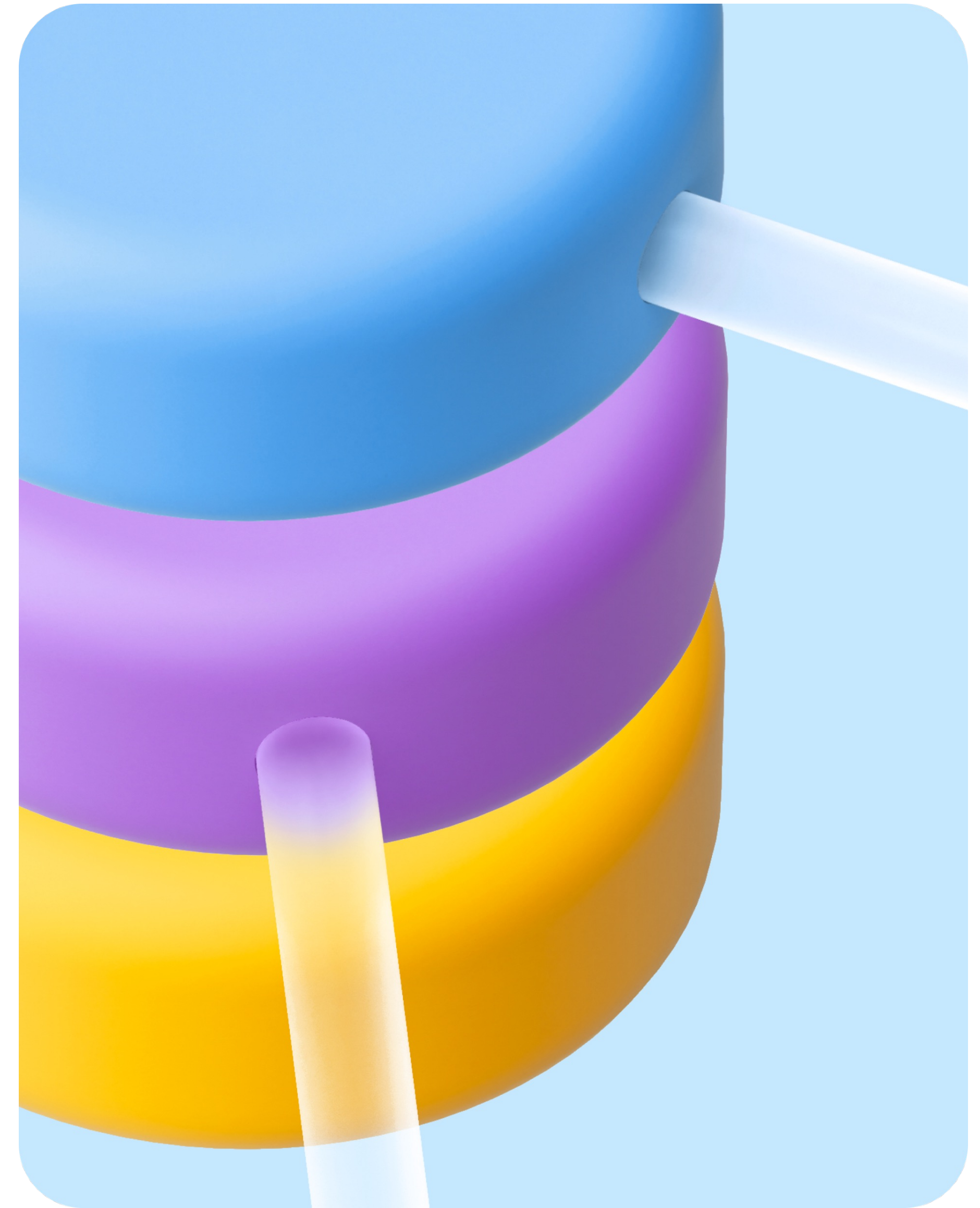
**Какие использовали методы и средства
инструментального мониторинга в продакшене**

3

**Как мы оцениваем эффект от потенциальных
улучшений с помощью теории массового обслуживания**

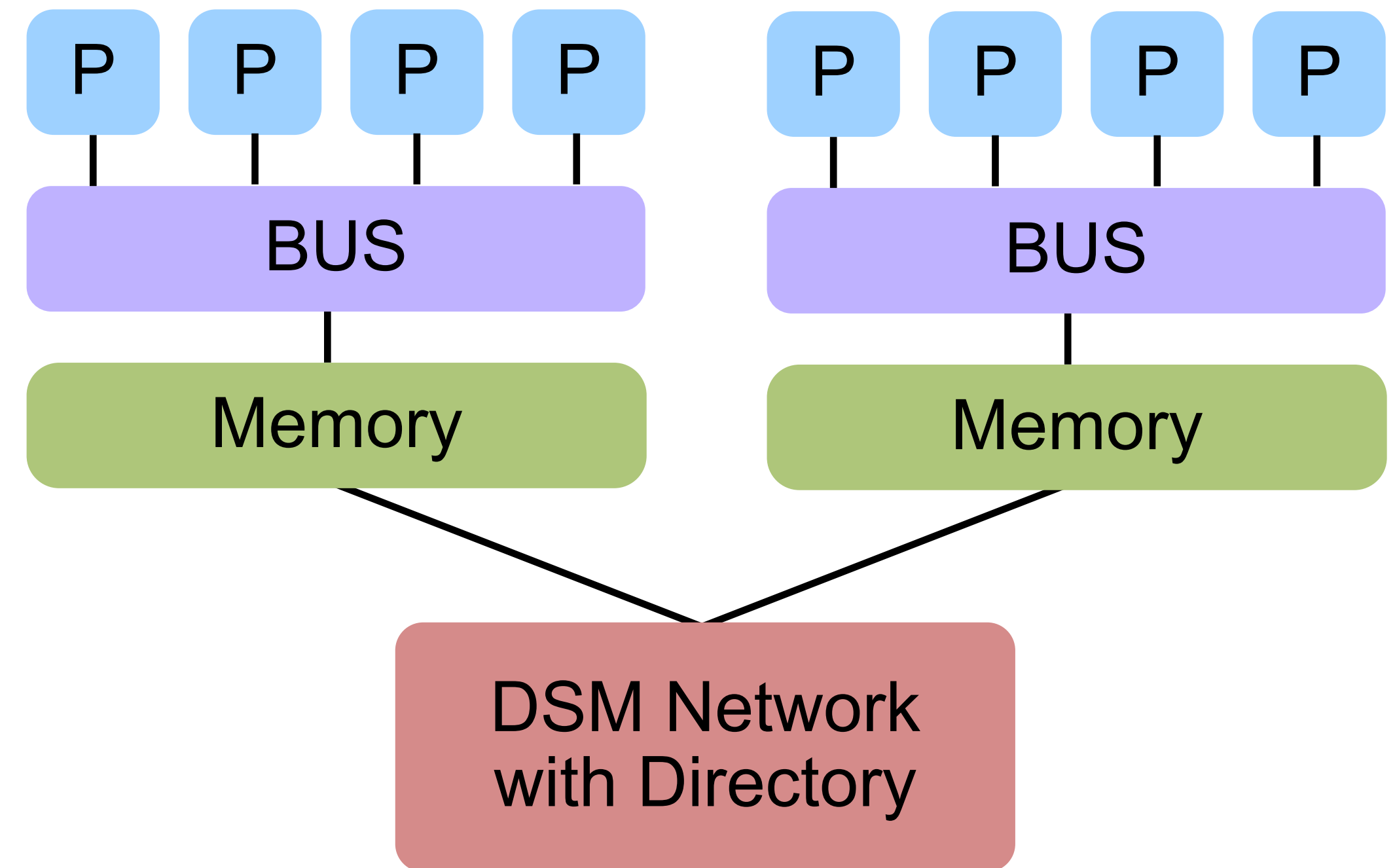
Улучшаем performance, не трогая код

02



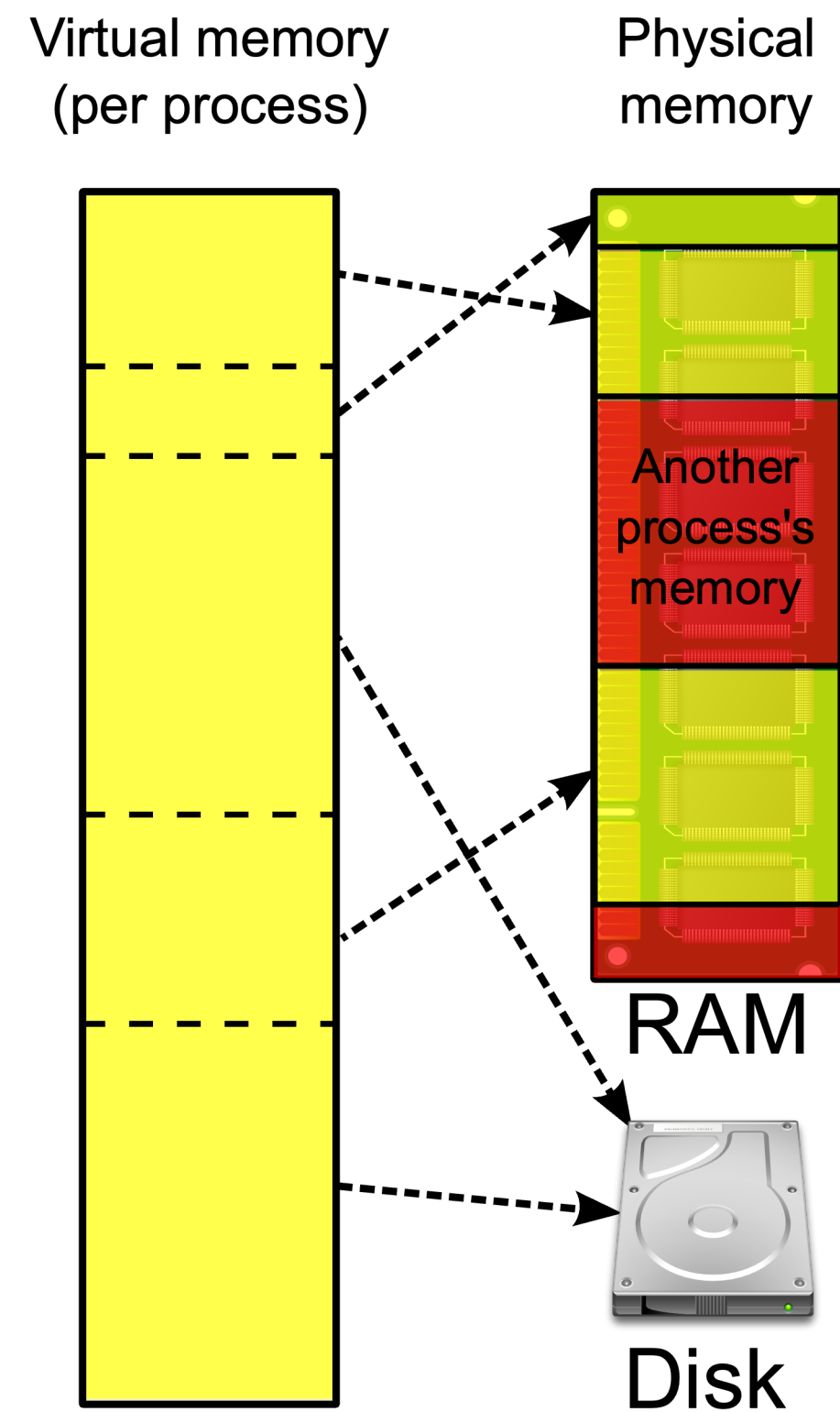
NUMA

- Данные могут быть локальны относительно друг друга, но лежать в «чужой» NUMA-ноде
- Поход в «чужую» NUMA-ноду очень дорог
- Если выставить affinity так, чтобы все потоки были на ядрах одной NUMA-ноды, можно получить огромный прирост производительности
- В случае YDB это **+20%** к пропускной способности!

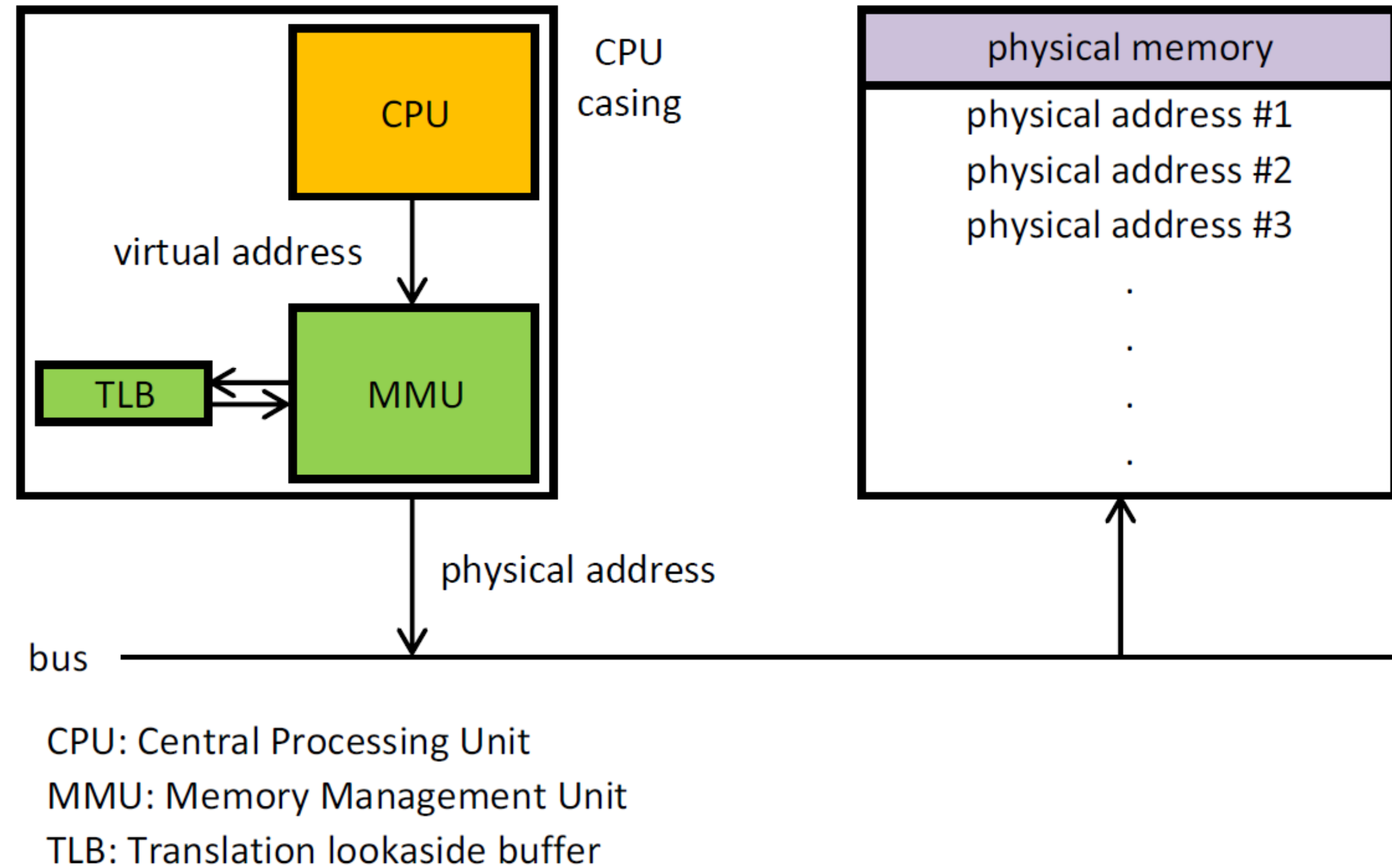


[Источник](#)

Виртуальная память и Huge pages



[Источник](#)



[Источник](#)

Виртуальная память и Huge pages

- Память организована в виде страниц: 4 KiB, 2 MiB, 1 GiB
- Маппинг виртуальных адресов хранится в RAM
- Обращение в память стоит дорого
- Translation lookaside buffer (TLB) кэширует адреса, его размер сильно ограничен
- Страницы большего размера – меньше TLB-промахов

YDB и huge pages

- TCMalloc поддерживает transparent huge pages
- Включение transparent huge pages даёт увеличение пропускной способности на **3-5%**
- Мало, но в больших инсталляциях это значительная экономия железа

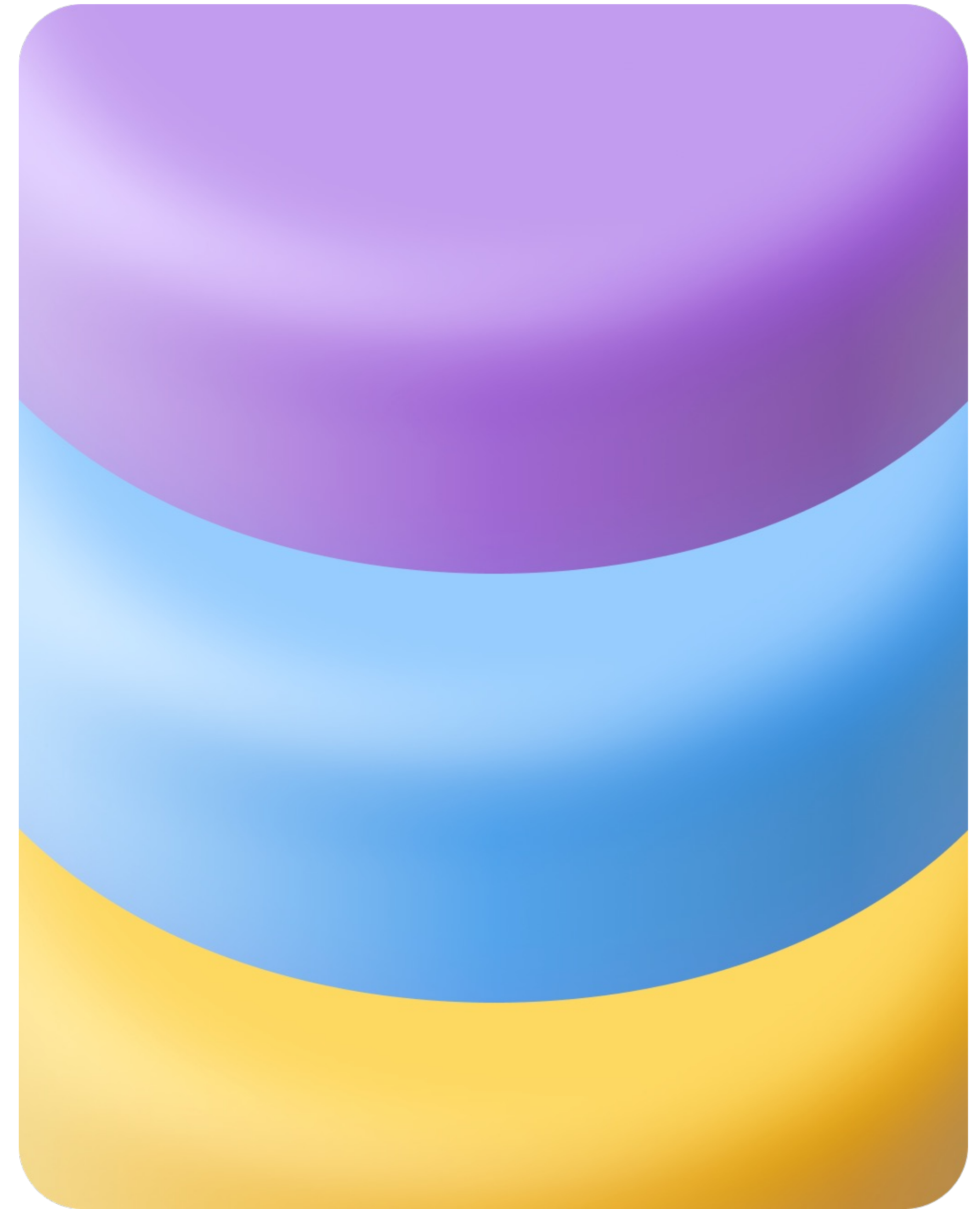
Магия компиляции

- Link-time optimization (LTO): **+1-3%** к пропускной способности
- Profile-guided optimization (PGO): большой потенциал, планируем использовать. Оцениваем в **+1-20%**



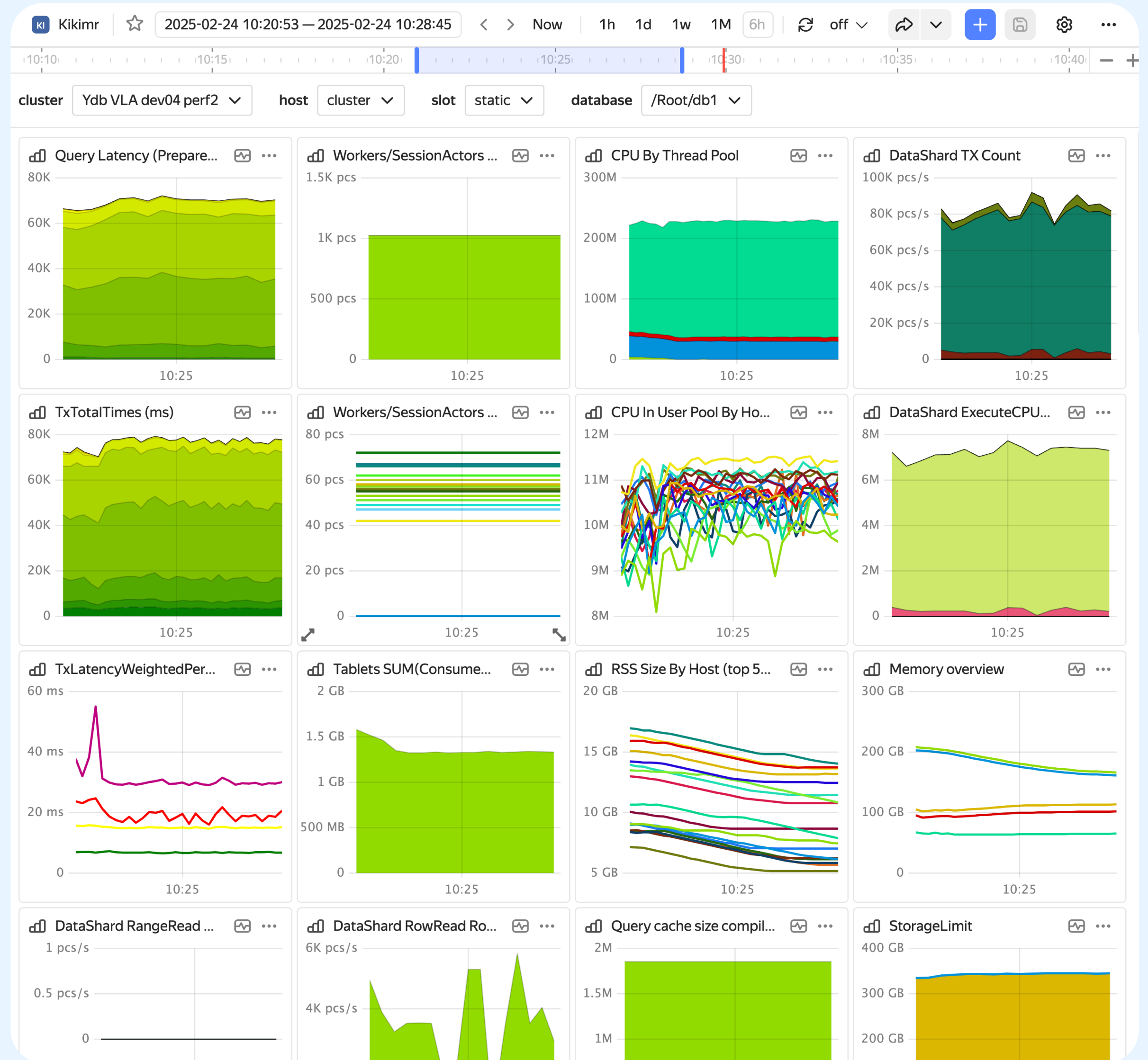
Метрики и распределенная трассировка

03



Метрики

- число активных запросов в каждой из частей системы
- latency отдельных компонент и системы в целом
- потребление ресурсов: диск, CPU, сеть, RAM
- распределение нагрузки по нодам
- сколько запросы читают строк / байтов
- и многое другое



Пример из кода: создание метрик

```
::NMonitoring::TDynamicCounterPtr Counters;  
::NMonitoring::TDynamicCounters::TCounterPtr InFly;  
::NMonitoring::TDynamicCounters::TCounterPtr Ok;  
::NMonitoring::TDynamicCounters::TCounterPtr Error;  
::NMonitoring::THistogramPtr LatencyMs;  
  
void Register() {  
    auto subgroup = Counters->GetSubgroup("request", Name);  
    InFly = subgroup->GetCounter("InFly", false);  
    Ok = subgroup->GetCounter("Ok", true);  
    Error = subgroup->GetCounter("Error", true);  
    LatencyMs = subgroup->GetHistogram("LatencyMs", GetLatencyHistogramBuckets());  
}
```

Пример из кода: использование

```
void Bootstrap() {  
    Counters->Register();  
    Counters->InFly->Inc();  
    AsDerived()->Start();  
}
```

```
void CompleteAndPassAway() {  
    Counters->Ok->Inc();  
    PassAway();  
}
```

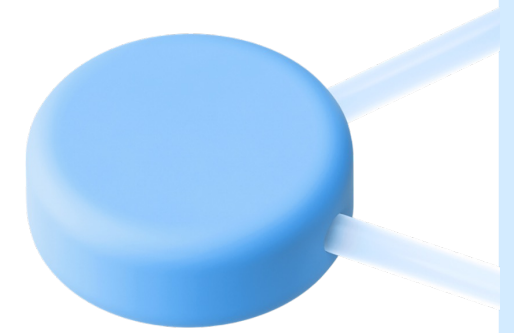
```
void FailedAndPassAway() {  
    Counters->Error->Inc();  
    PassAway();  
}
```

Как это работает

- Метрики шарятся между объектами, выполняемыми параллельно
- Внутри атомики, лок только при регистрации новых метрик
- Отгрузка метрик работает параллельно
- OpenTelemetry
- Можно использовать Grafana, но внутри у нас своя система мониторинга на платформе YDB
- В нашем случае нет видимого влияния на производительность
- Метрики всегда включены в проде

Метрик много не бывает

Даже у маленького кластера из 3 серверов
(5 процессов YDB на сервер):



≈ 800 000

метрик

Запись 80 000

метрик в секунду

**Бесконечно можно смотреть на три вещи:
как горит огонь, как течёт вода и на метрики YDB**



Распределенная трассировка

- Метрики агрегируются: как понять, что происходило с отдельным запросом?
- Запрос может затрагивать большое число хостов и процессов YDB – обычной трассировки не достаточно

- YDB поддерживает распределенную трассировку
- OpenTelemetry / Jaeger

Trace structure

Critical path

1 Specify the query parameters, e.g., span.status = error

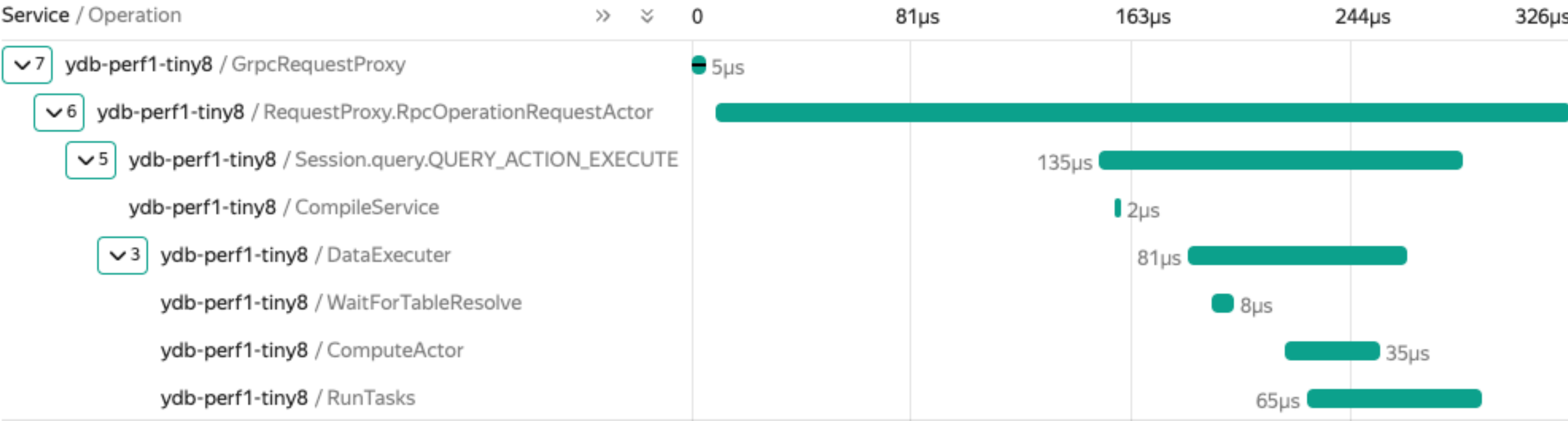
Execute

Query language

Spans with errors (0)

Spans on critical path (1)

Show only found results



Влияние на производительность

- В основе реализации наш аналог [LTTng](#)

- В продакшене используем семплирование: 1 трейс в минуту с каждой ноды

- Трейс отдельных событий почти не даёт оверхеда, но событий много

- В больших инсталляциях это порядка 10 000 трейсов в минуту

Недостатки трейсинга

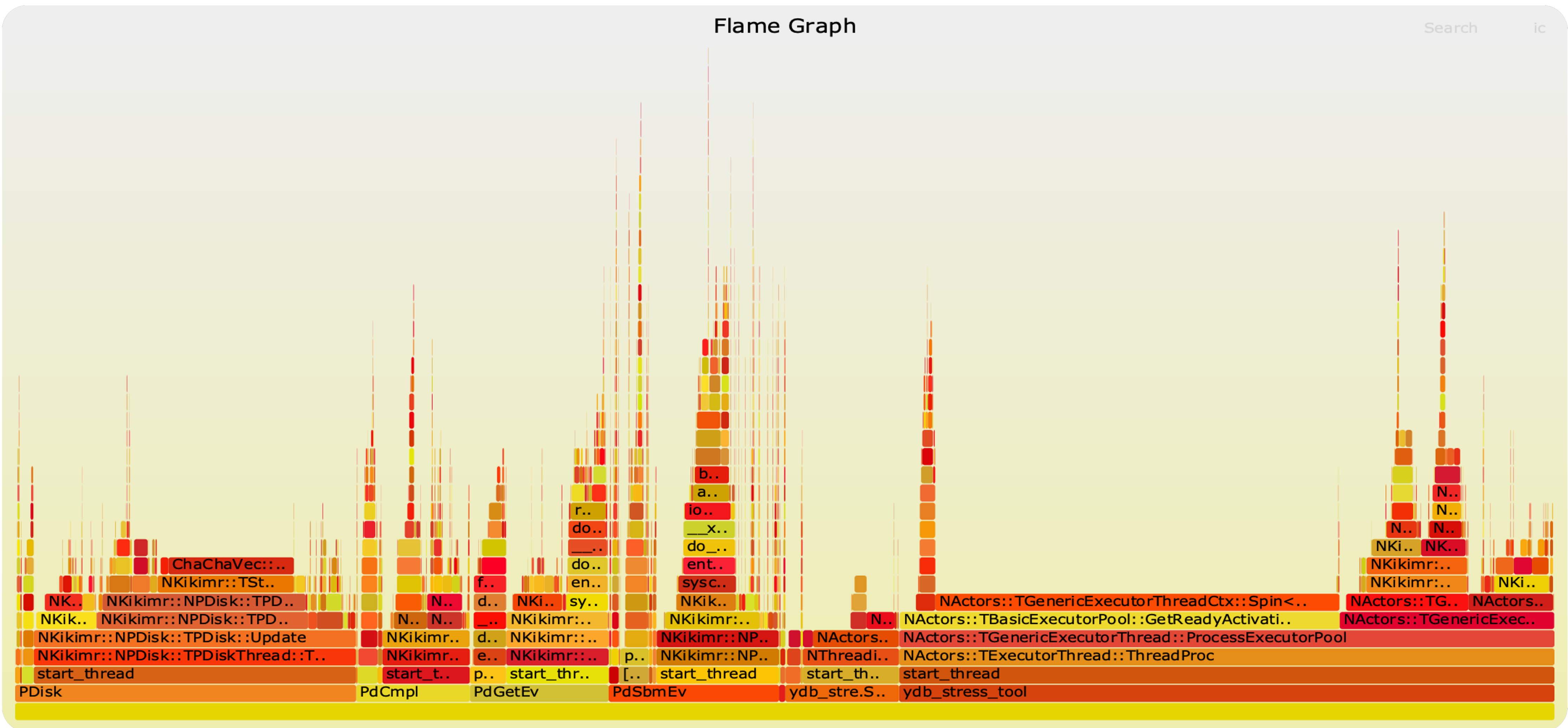
- При нагрузочном тестировании тяжело анализировать задержки на p99 и выше: аутлайеры в разных частях системы, нужен дополнительный анализатор трейсов

- Трейс показывает, сколько **времени** мы находились внутри компоненты, но не показывает, сколько **реально выполнялись**: очереди перед компонентами, снятие с CPU

Флеймграфы, внутренний ring service и eBPF



Еще одна вещь для долгого созерцания



Флеймграф недели

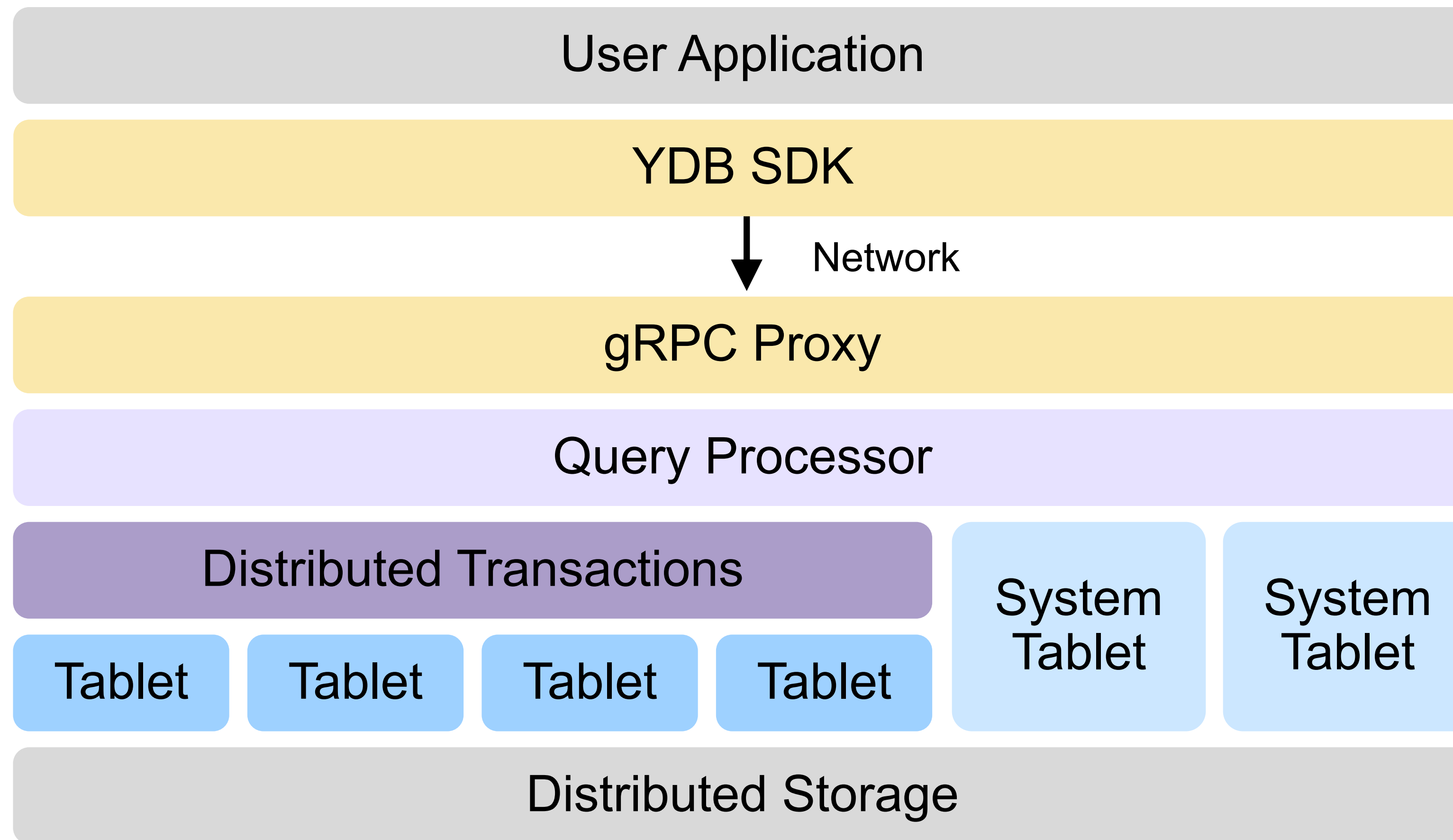
- На протяжении месяцев каждую неделю снимали флеймграф с системы под нагрузкой

- Постепенно достигли кратного увеличения пропускной способности

- Каждый раз находили что-то интересное: например, деструкторы некоторых объектов вдруг занимали много процессорного времени

- И просмотр флеймграфов перестал помогать – что дальше?

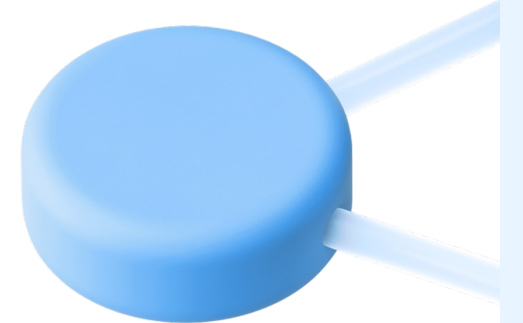
Архитектура YDB: многослойность в моде



Внутренний ring-сервис

- 1** Добавили в каждый слой поддержку специального ring-запроса
- 2** Поддержали отправку этих запросов через ydb CLI
- 3** Используем для нагрузочного тестирования отдельных слоёв

- 4** Запускаем вместе со стандартными бенчмарками, чтобы упростить поиск узких мест, которые им мешают



- 5** Пока только начали находить интересные вещи

eBPF и BCC

Используем BPF Compiler Collection (BCC):

- `biolateness`: задержки дисков
- `runqslower`: задержки в шедулинге потоков
- `offcpu / offcputime / wakeuptime`: ПОИСК причин и оценка времени, сколько поток НЕ МОГ ВЫПОЛНЯТЬСЯ

runqslower: переподписка по CPU

```
sudo python3 /usr/share/bcc/tools/runqslower -p 3501234 1000 -P
```

Tracing run queue latency higher than 1000 us

TIME	COMM	TID	LAT(us)	PREV COMM	PREV TID
09:29:33	PdTrim	3501323	123883	PdGetEv	3501321
09:29:33	PdGetEv	3501321	4927	network poller	3501251
09:29:36	PdSbmEv	3501320	25299	ydbd.User	3501249
09:29:36	network poller	3501251	1431	ydbd.System	3501246
09:29:39	PdSbmEv	3501320	111226	PdGetEv	3501321
09:29:39	PdGetEv	3501321	4923	swapper/4	0
09:29:39	ydbd.Scheduler	3501257	3025	PdGetEv	3501321
09:29:40	network poller	3501251	6013	swapper/7	0
09:29:41	network poller	3501251	2271	ydbd.User	3501249
09:29:41	ydbd.Scheduler	3501257	5583	swapper/5	0
09:29:42	ydbd.Scheduler	3501257	4024	PdGetEv	3501321
09:29:43	PdSbmEv	3501320	100573	PdGetEv	3501321
09:29:43	PdGetEv	3501321	4924	swapper/2	0
09:29:45	PdSbmEv	3501320	112318	PdGetEv	3501321

```
sudo python3 /usr/local/share/bcc/tools/offcputime -p 366805 -m 200 -U 10
```

```
syscall
```

```
absl::lts_20240722::base_internal::SpinLock::SlowLock()
```

```
tcmalloc::tcmalloc_internal::(anonymous namespace)::do_free_pages(void*,  
__libc_free
```

```
NKikimr::NGRpcService::(anonymous namespace)::TExecuteQueryRPC::~~TExecuteQueryRPC()
```

```
...
```

```
- ydbd.User (366958)
```

```
1545
```

```
syscall
```

```
absl::lts_20240722::base_internal::SpinLock::SlowLock()
```

```
tcmalloc::tcmalloc_internal::(anonymous namespace)::do_free_pages(void*,  
__libc_free
```

```
...
```

```
- ydbd.System (366957)
```

```
1975
```

```
syscall
```

```
absl::lts_20240722::base_internal::SpinLock::SlowLock()
```

```
tcmalloc::tcmalloc_internal::CentralFreeList::InsertRange(absl::lts_20240722::Span<void*>)  
memalign
```

```
NDetail::TRcBufInternalBackend::Allocate(unsigned long, unsigned long, unsigned long)
```

```
TRcBuf::Uninitialized(unsigned long, unsigned long, unsigned long)
```

```
TRcBuf::GetDataMut(unsigned long, unsigned long)
```

```
NActors::TInputSessionTCP::ReceiveData()
```

```
...
```

```
- ydbd.IC (366969)
```

```
2123
```

offcrutime: нашли лок в TCMalloc

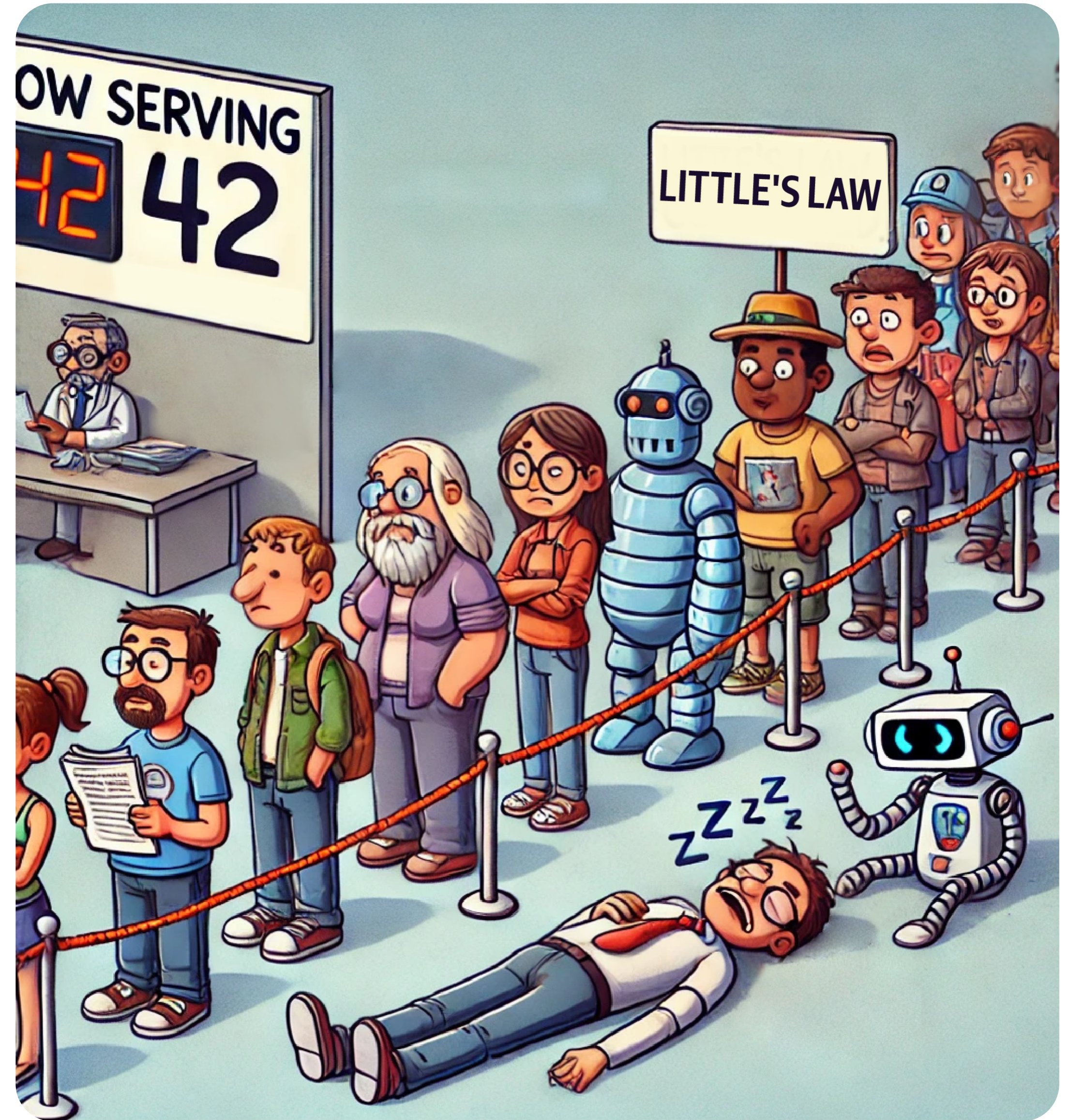
**Аллокация
и деаллокация
происходят
на разных CPU**

**Кэш TCMalloc
per-CPU и быстро
становится пустым**

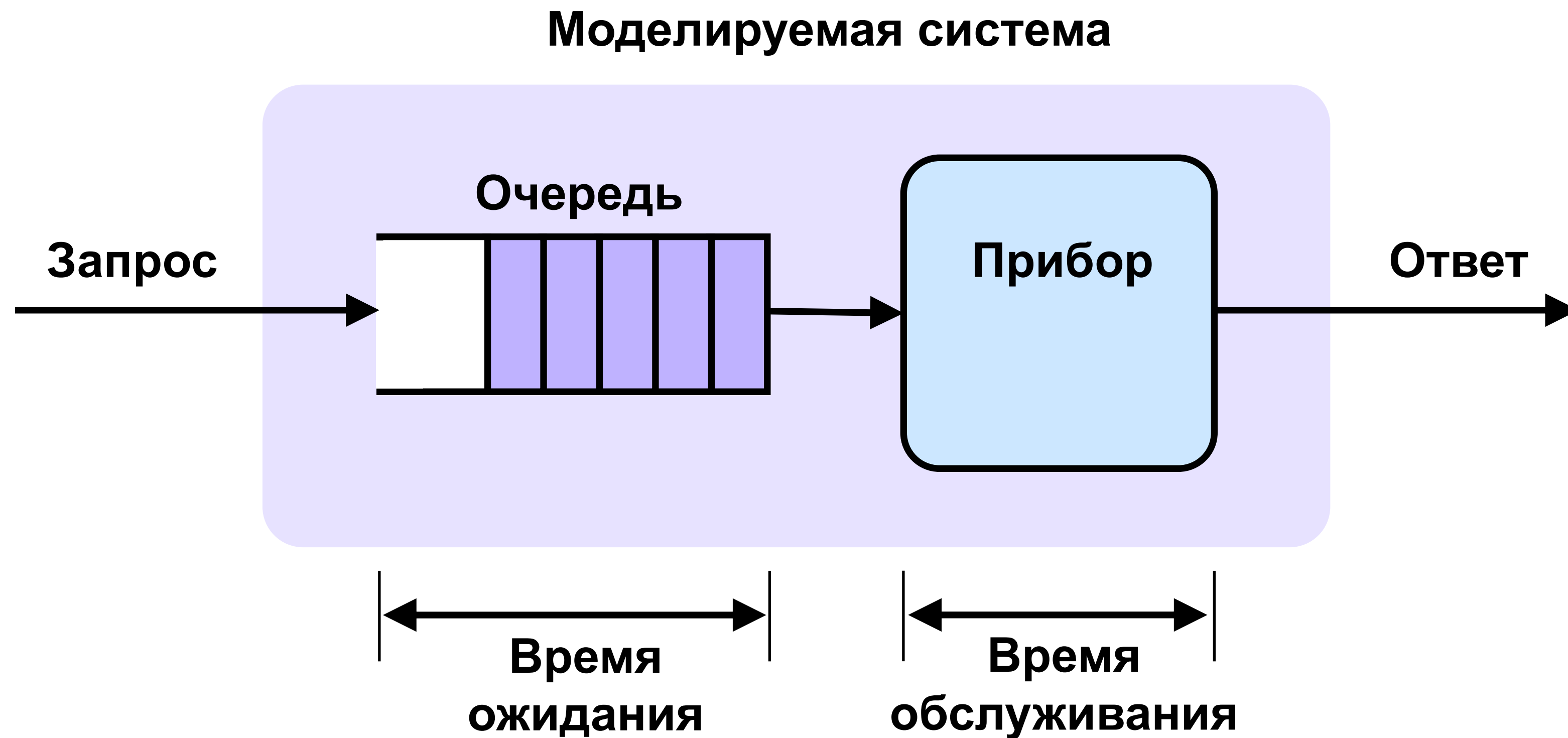
**Возврат памяти
в центральный
кэш через почти
глобальный лок**

Теория массового обслуживания в действии

05



Простая модель сложной системы



Простая и выразительная модель для описания работы:

CPU

**Отдельных программных
компонент**

Жёсткого диска

**Одного сервера или
целого кластера**

**Парковок и касс
в супермаркете**

Отвечает на важные вопросы

1

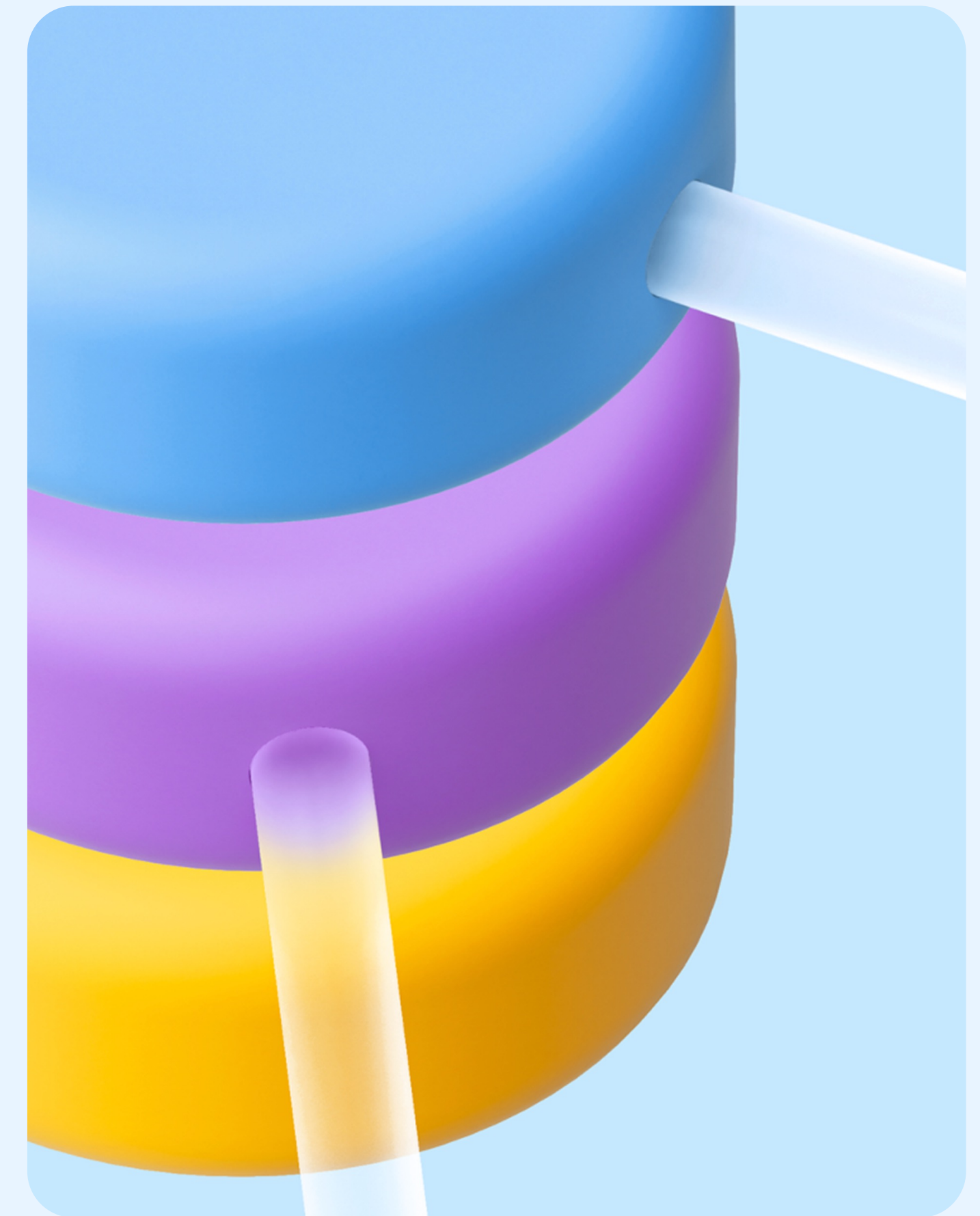
Как изменится время ответа при добавлении CPU / потока?

2

Насколько увеличится время ответа при увеличении нагрузки в несколько раз?

3

При каких значениях нагрузки время ответа на p99 не превысит 50 мс?



Чем это лучше нагрузочного тестирования?

1 Код, с помощью которого достигается распараллеливание, не всегда написан. И написать его бывает сложно и долго

3 Нет нужного железа (например, более быстрого диска)

2 Нет нужного количества железа (1000 серверов vs. 2000)

4 Удобно экспериментировать, меняя разные части системы

Два основных подхода

1

Аналитическое (математическое) моделирование:

- очень сложное
- удобно работать с простыми распределениями

2

Имитационное моделирование:

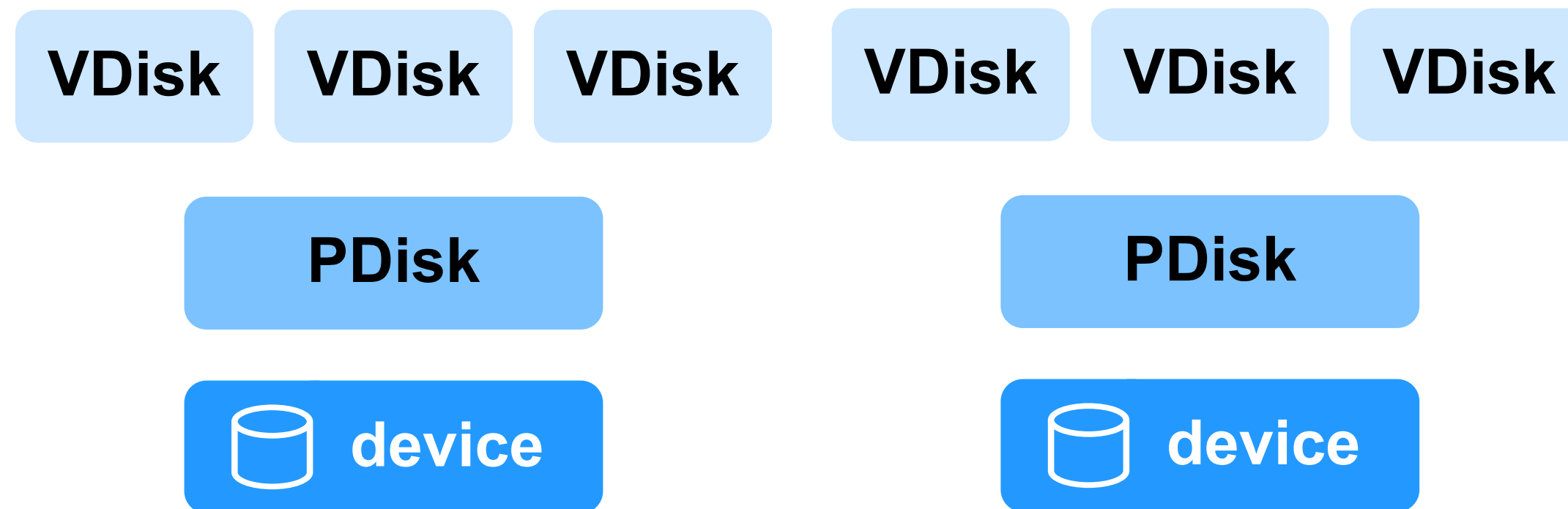
- прагматичнее
- требует специализированного софта и умения им пользоваться
- достаточно сложно задавать и менять логику сложных моделей

**«Ты ж программист!», –
сказал мне
мой руководитель**

и предложил сделать **дискретно-событийный симулятор** на C++
вместо имитационного
моделирования



Нода распределенного хранилища



PDisk обслуживает несколько VDisk'ов:

- работает напрямую с устройством
- хранит чанки фиксированного размера
- оптимизирован для записи лога
- использует честный шедулер, разделяющий полосу диска между VDisk'ами
- шифрует данные

Быстрый NVMe и медленный PDisk?

1

Нагрузка: запустили `fio (aio)` с `queuedepth = 64` и размером записи 8 KiB (у нас нагрузка на диск значительно меньше)

2

Запустили `biolatency`, чтобы узнать, как быстро работает диск

Быстрый NVMe и медленный PDisk?

```
sudo biolateness-bpfcc -Q -D 120 1
```

```
disk = b'nvme0n1'
```

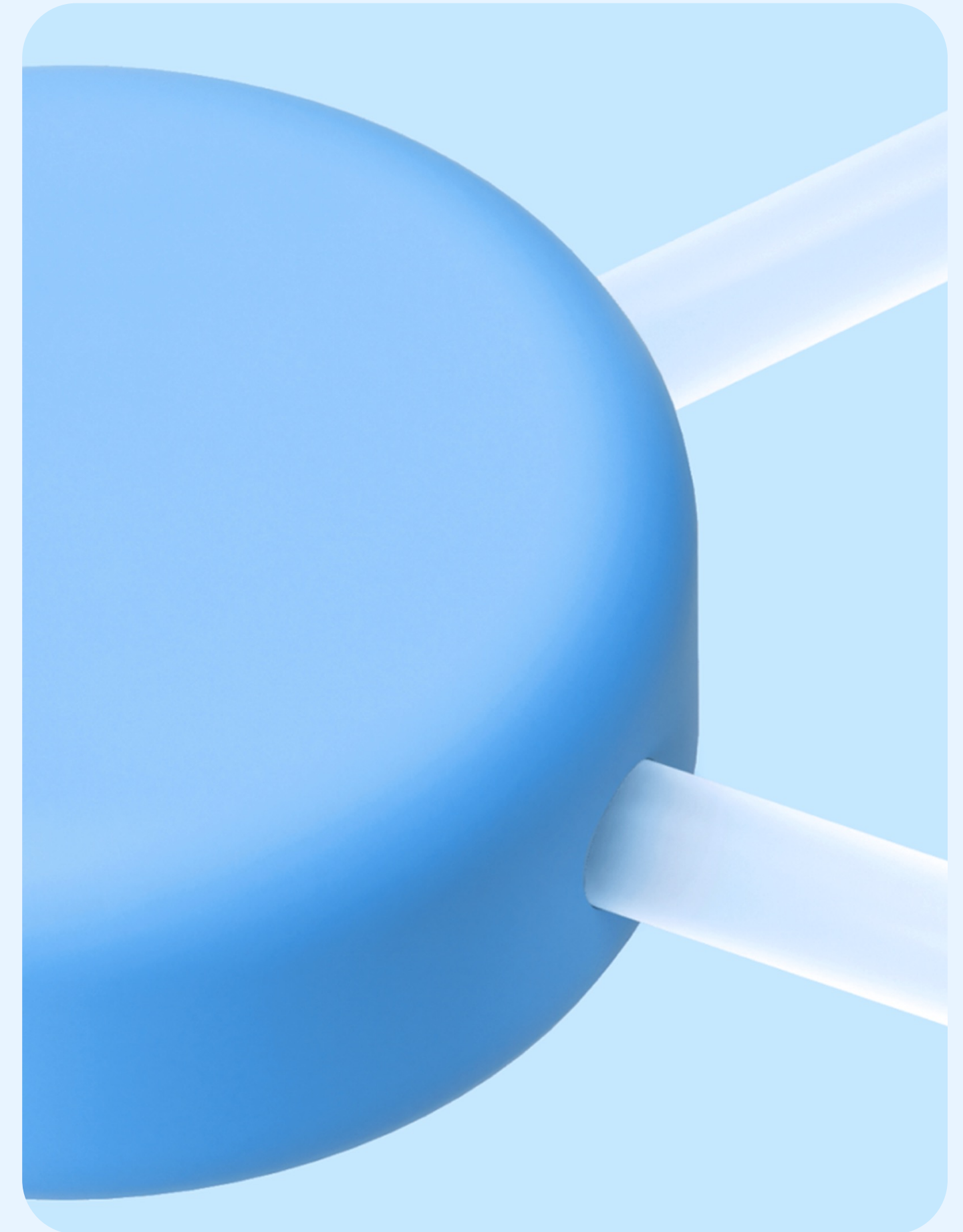
usecs	: count	distribution
0 -> 1	: 0	
2 -> 3	: 0	
4 -> 7	: 0	
8 -> 15	: 3044812	*****
16 -> 31	: 12545723	*****
32 -> 63	: 2033894	*****
64 -> 127	: 1166428	***
128 -> 255	: 259422	
256 -> 511	: 966	
512 -> 1023	: 1323	

Быстрый NVMe и медленный PDisk?

1 Даже под большой нагрузкой NVMe выполняет почти все запросы за пару десятков микросекунд

2 По метрикам PDisk у нас уже на p10 150 мкс

3 В PDisk ничего долгого не делаем. Что за ерунда?



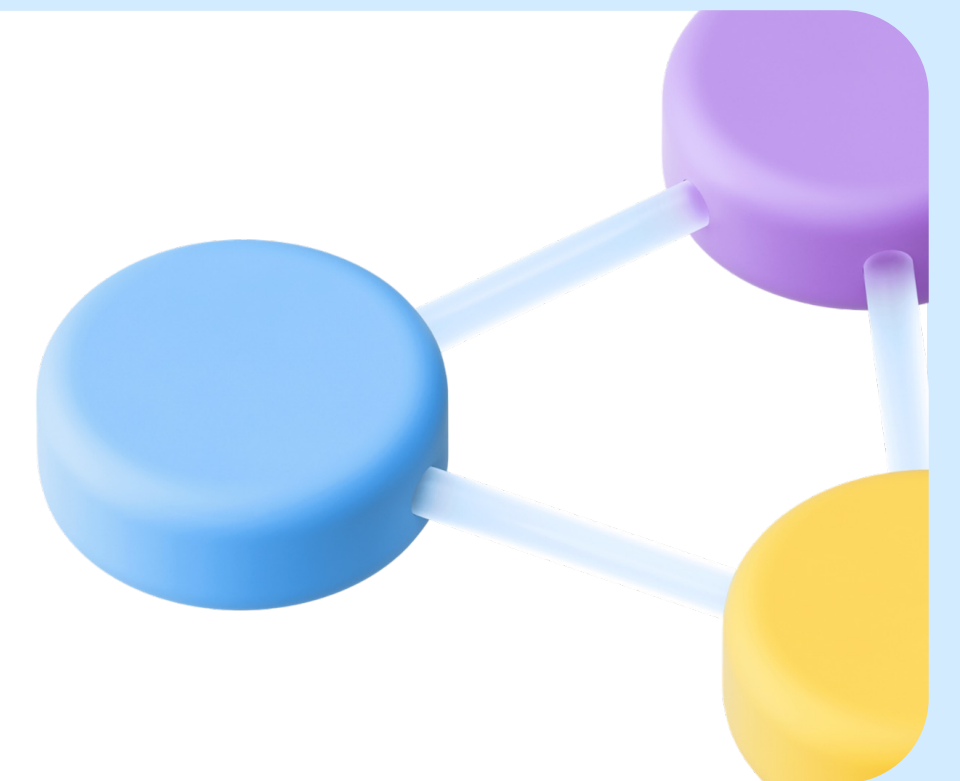
Нам нужна модель PDisk'a

1

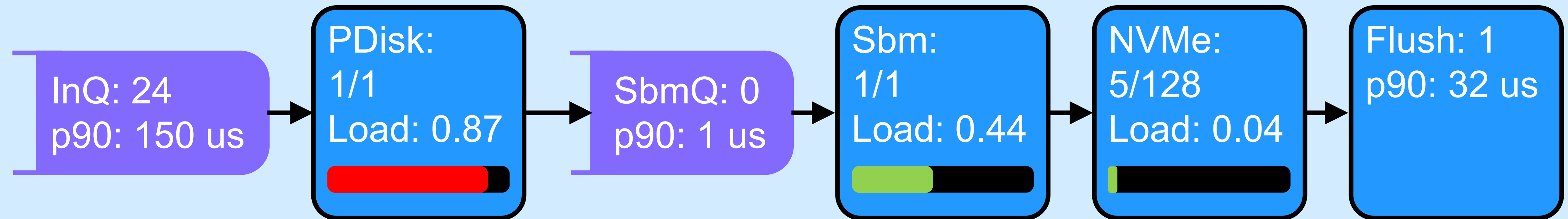
По метрикам, трейсам и с помощью BCC собрали информацию об очередях и времени выполнения частей PDisk'a и наших NVMe под нагрузкой

2

Построили имитационную модель



Компоненты модельного PDisk'a



Перед каждым компонентом есть очередь.

1. «PDisk» разбирает входную очередь:

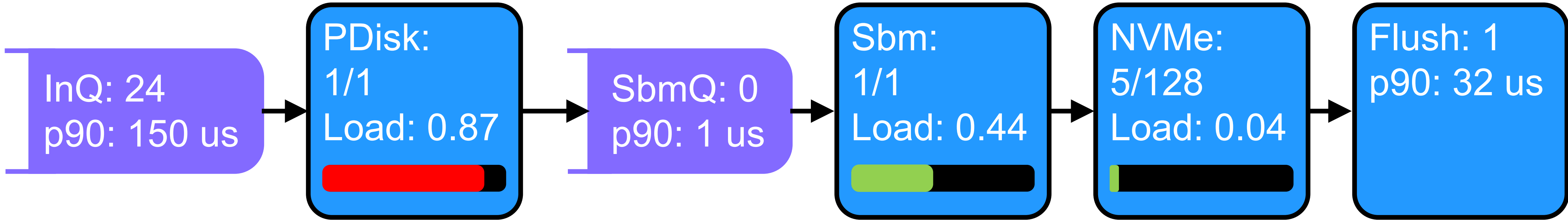
- чексуммы
- шифрование
- честный шедулер

2. «Sbm» отправляет данные на запись через libAIO

3. NVMe диск (параллельность до 128 операций)

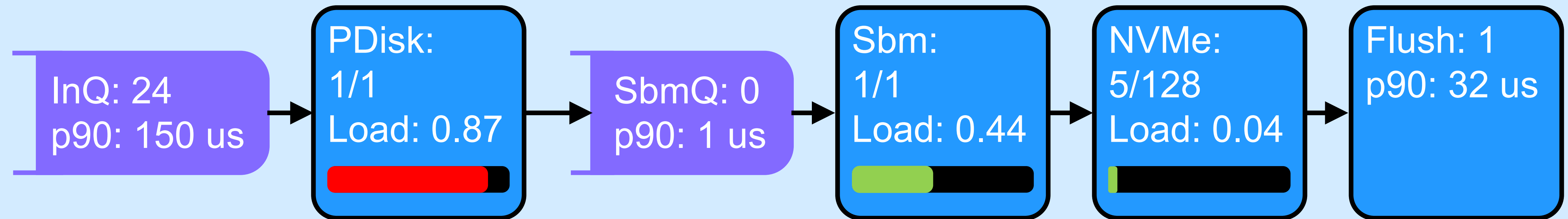
4. «Flush»: запись не завершается, пока не завершились все предыдущие

Симуляция PDisk



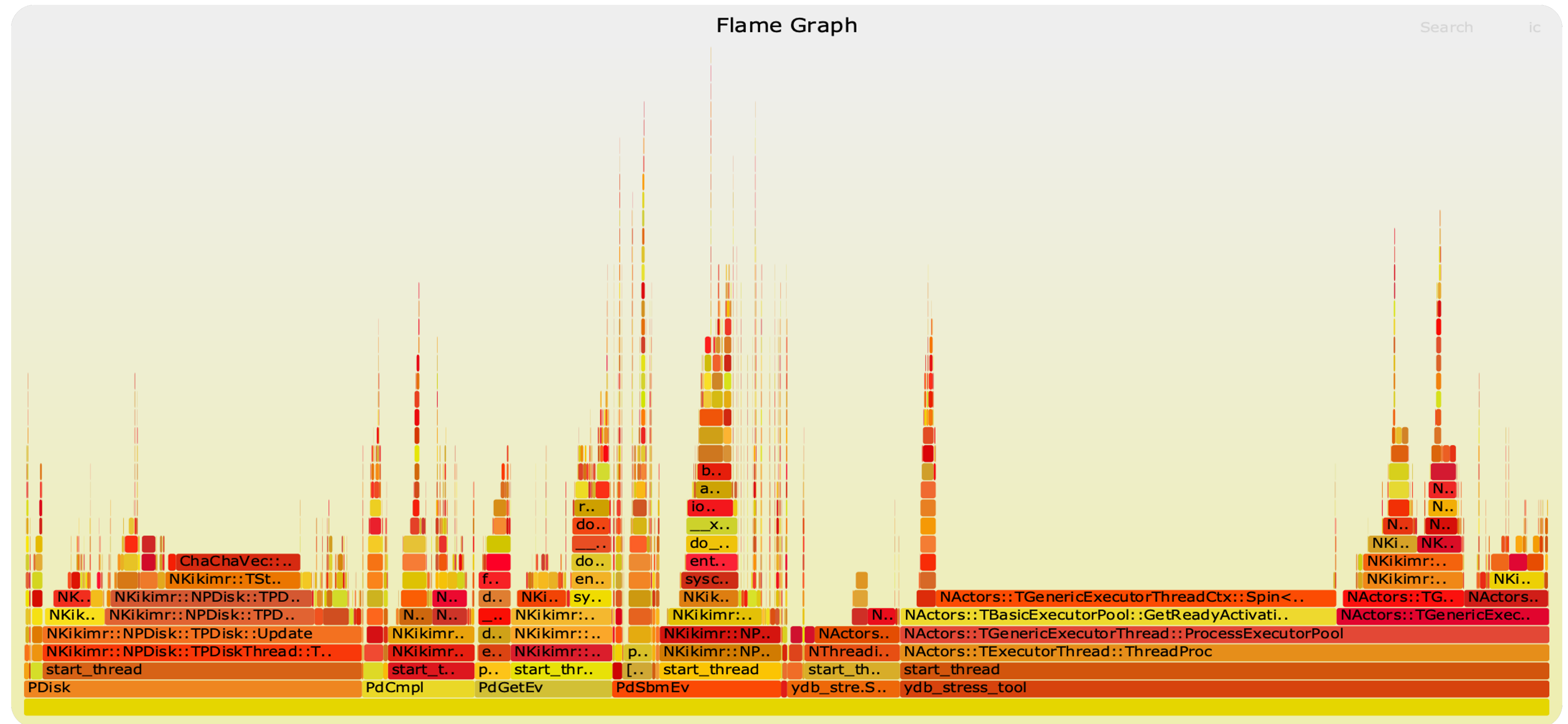
TimePassed: 59.20 s, Events: 9533596, AvgRPS: 161040
p10: 160 us, p50: 190 us, p90: 250 us, p99: 300 us, p100: 4250 us

Что мы узнали

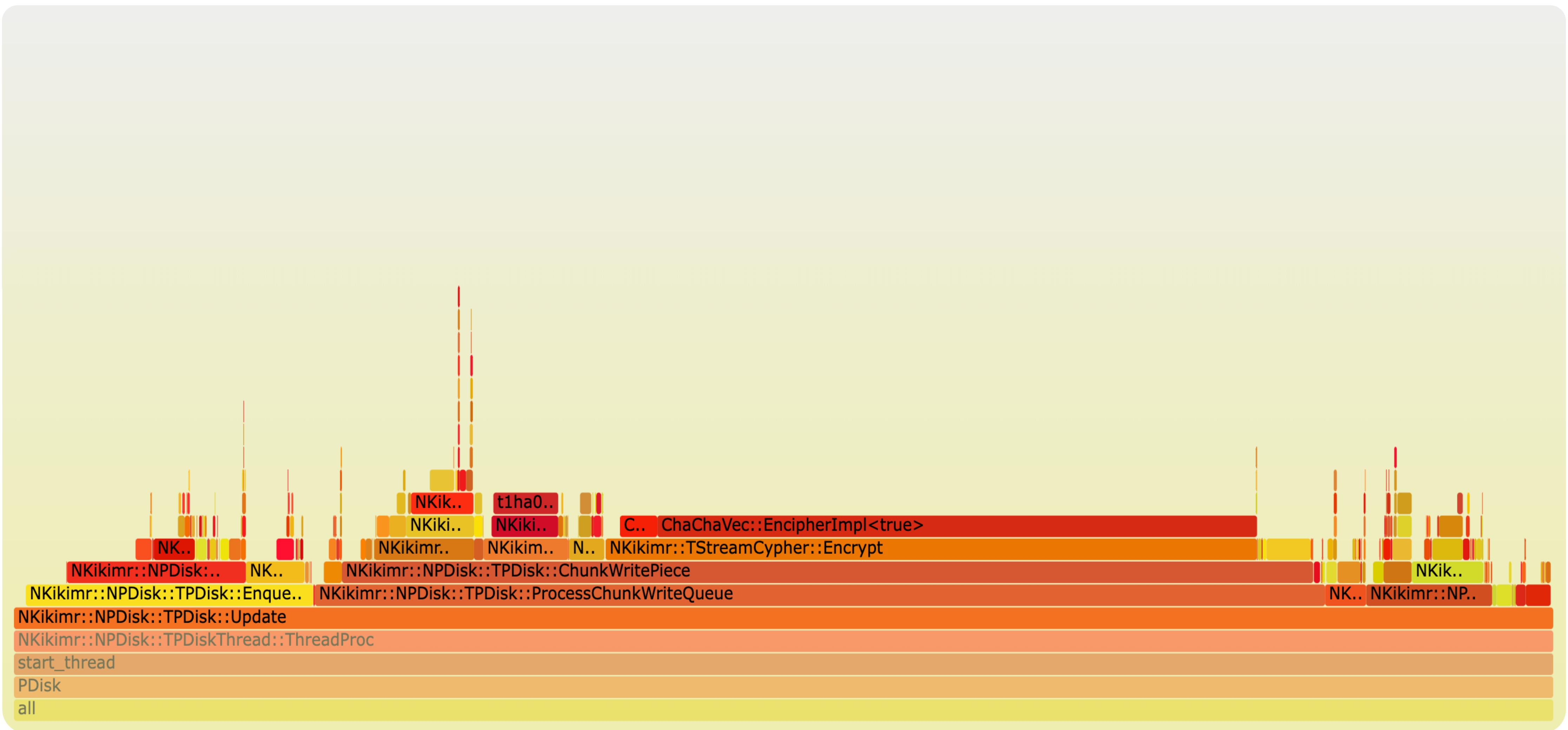


1. Результаты модели сошлись с метриками
2. При равномерном поступлении запросов все работает быстро
3. При бёрстах накопленная очередь InQ не разгребается
4. PDisk – узкое место. Время ожидания в InQ вносит основной вклад в общее время выполнения

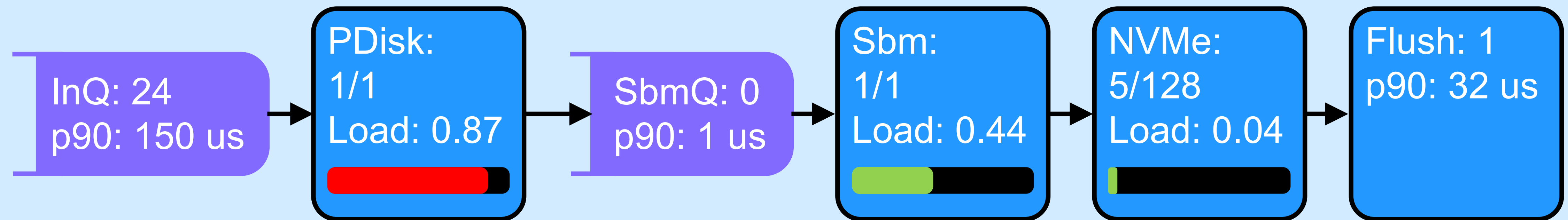
Возвращение флеймграфов: чем занят PDisk



Возвращение флеймграфов: чем занят PDisk



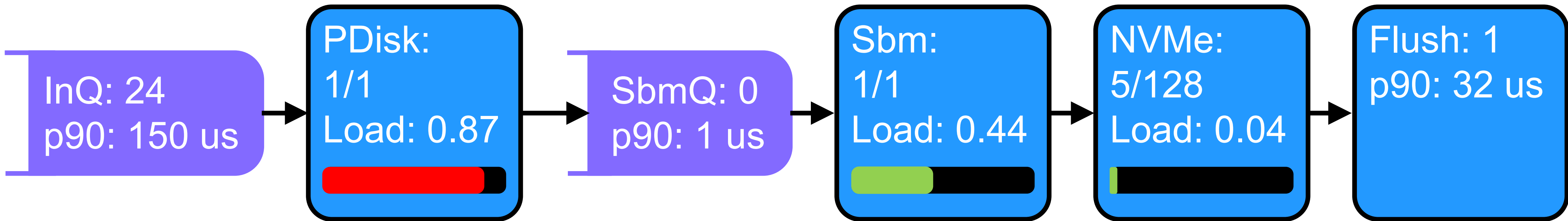
Улучшаем PDisk



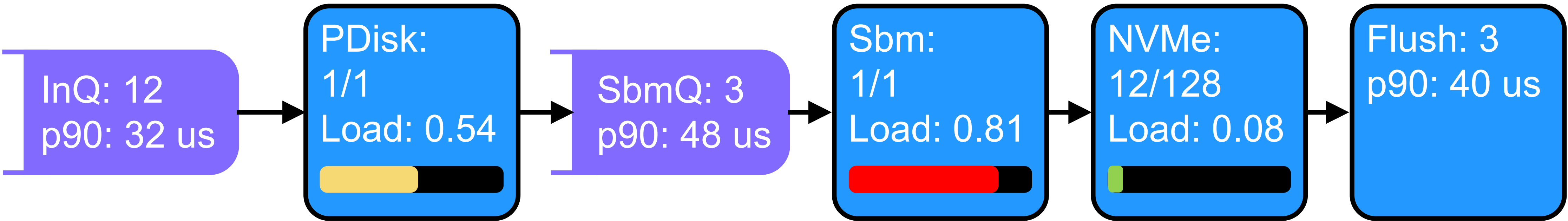
Разные опции ускорения PDisk:

- AVX2 для шифрования
- Не шифровать: чуть быстрее AVX2 и работает на любом железе
- Zero сору, раз не шифруем (трудозатрано)

До и после улучшения PDisk

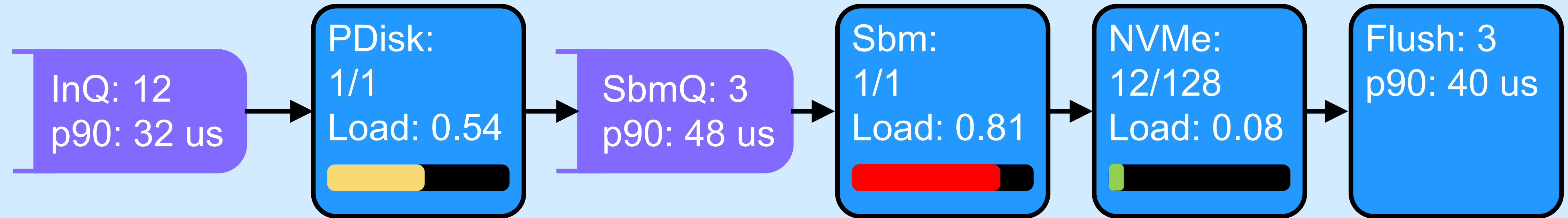


TimePassed: 59.20 s, Events: 9533596, AvgRPS: 161040
p10: 160 us, p50: 190 us, p90: 250 us, p99: 300 us, p100: 4250 us



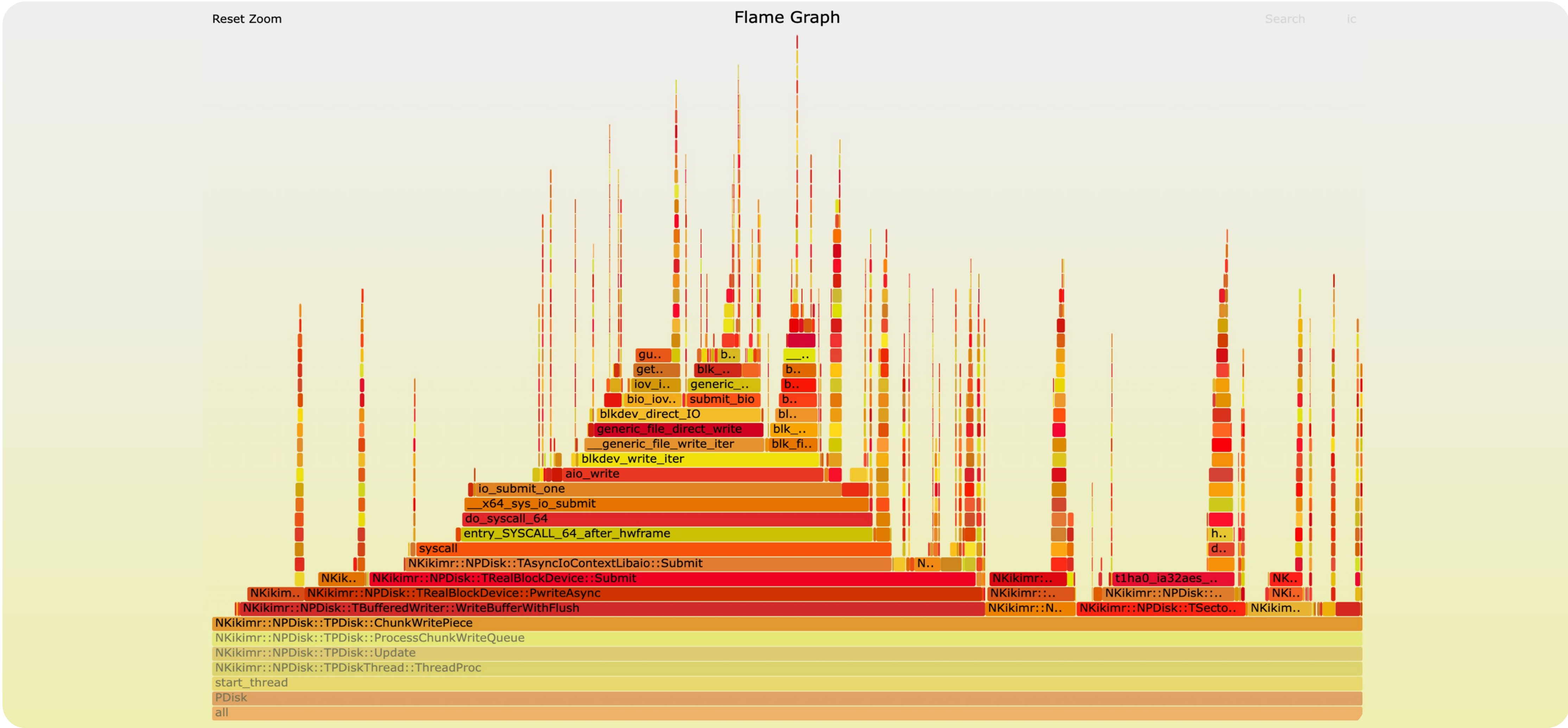
TimePassed: 135.20 s, Events: 37264646, AvgRPS: 275625
p10: 70 us, p50: 100 us, p90: 130 us, p99: 200 us, p100: 4250 us

Модель после улучшения PDisk



1. В два раза быстрее на всех персентилях по p99
2. NVMe не догнали
3. Sbm – новое узкое место
4. Хорошо, что не стали пилить ZeroCору до моделирования

Чем занят Sbm?



Перспективы улучшения Sbm

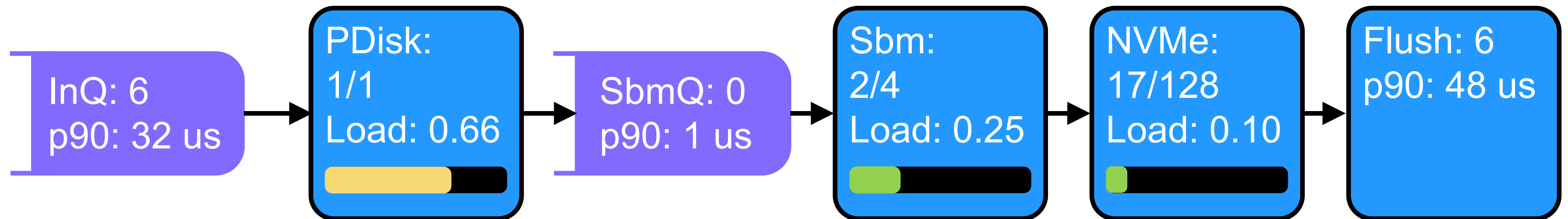
1 Sbm через `libAIO` отправляет запросы *по одному* на диск, всего один системный вызов, а задержки, как у самого диска!

2 Попробовали использовать в реальном Sbm батчевание запросов в `libAIO` – не помогло

3 Перейти на `uring` не можем

4 Добавление потоков в Sbm решает проблему, но `PDisk`’ов много и требуется реализация общего Sbm’а, чтобы избежать переподписки по числу потоков

Улучшенный Sbm в модели



TimePassed: 41.60 s, Events: 14546629, AvgRPS: 349678
p10: 54 us, p50: 70 us, p90: 110 us, p99: 180 us, p100: 4250 us

Реализация модели на C++

1

Для визуализации используем фреймворк [Arctic Engine](#):

- простой и функциональный
- кроссплатформенный
- лицензия MIT

2

Библиотека и модель сделаны быстро на коленке:
[queue simulations](#)

Код модели (1)

```
void SetupCurrentPdiskModel(ClosedPipeLine &pipeline) {  
    constexpr size_t startQueueSize = 32;  
  
    constexpr size_t pdiskThreads = 1;  
    constexpr double pdiskExecTime = 5 * Usec;  
  
    constexpr size_t sbmThreads = 1;  
    constexpr double sbmExecTime = 2 * Usec;  
  
    constexpr size_t NVMeInflight = 128;  
    PercentileTimeProcessor::Percentiles diskPercentilesUs = {  
        {16.47, 12 * Usec},  
        {87.26, 25 * Usec},  
        {99.7, 50 * Usec},  
        {99.992, 100 * Usec},  
        {99.9968, 200 * Usec},  
        {100, 4000 * Usec},  
    };  
};
```

Код модели (2)

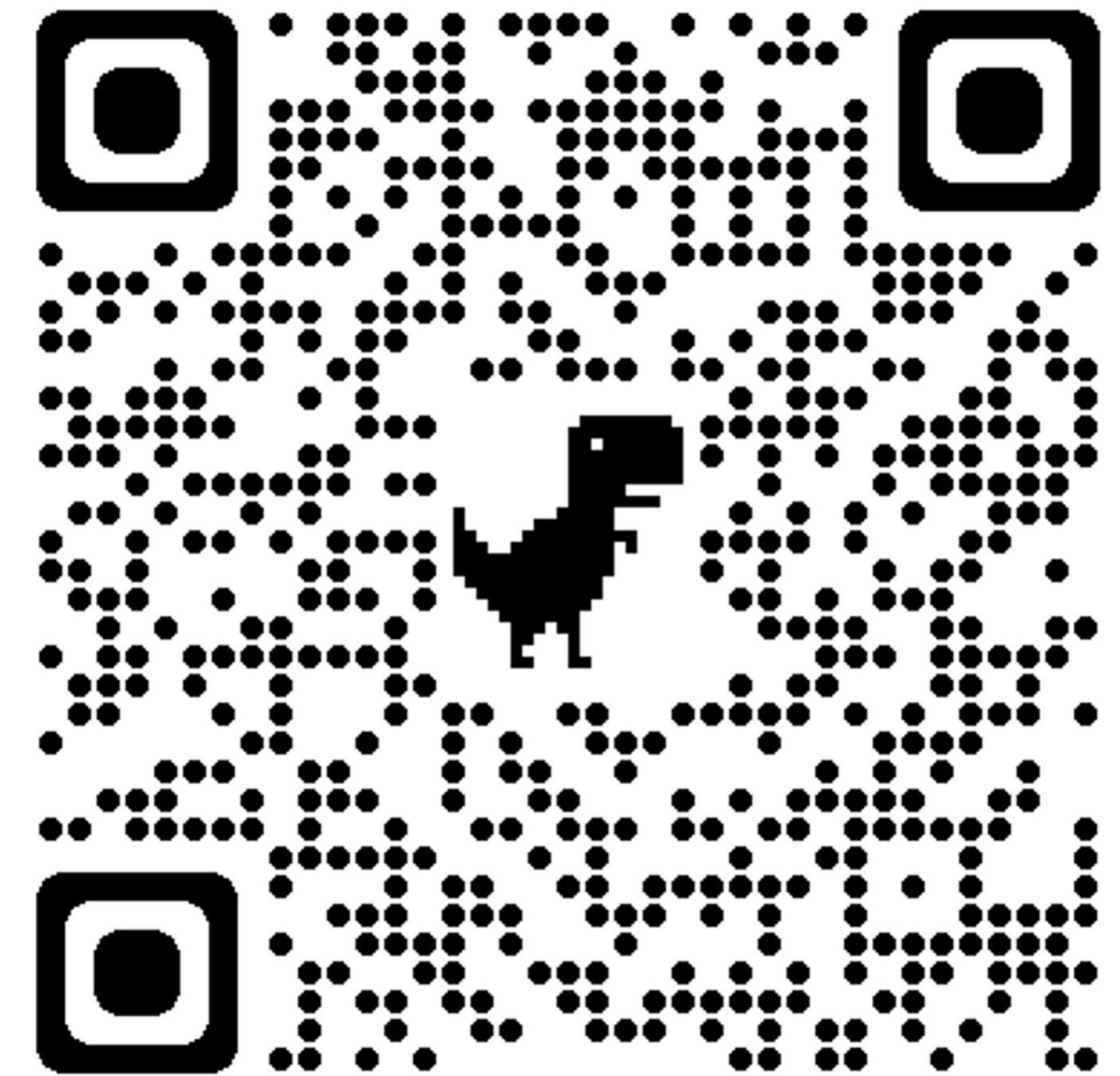
```
constexpr size_tvoid SetupCurrentPdiskModel(ClosedPipeLine &pipeline) {  
    ...  
  
    pipeline.AddQueue("InQ", startQueueSize);  
    pipeline.AddFixedTimeExecutor("PDisk", pdiskThreads, pdiskExecTime);  
    pipeline.AddQueue("SbmQ", 0);  
    pipeline.AddFixedTimeExecutor("Sbm", smbThreads, smbExecTime);  
    pipeline.AddPercentileTimeExecutor("NVMe", NVMeInflight, diskPercentilesUs);  
    pipeline.AddFlushController("Flush");  
}
```

Осторожно!



Заключение

- Производительность можно значительно улучшить за счет системных настроек, не меняя код
- Не существует серебряной пули для поиска узких мест: требуется совокупность методов от метрик, флеймграфов и трейсов до специальных заглушек в коде
- Теория массового обслуживания рулит



**блог, сообщество,
презентации и записи
выступлений, вакансии**



YDB



**C++
Russia**